API Security Risk Analysis Report

1. Introduction

APIs (Application Programming Interfaces) are widely used in modern applications such as mobile apps, SaaS platforms, dashboards, and integrations between services. However, insecure APIs can expose sensitive data, allow unauthorized access, or cause system abuse.

This report analyzes the security risks of a public demo API using safe and ethical testing methods.

The analysis was performed using **Postman** and browser developer tools to inspect API responses, authentication mechanisms, and headers.

---

2. API Information

API Name: JSONPlaceholder
Base URL:

```
https://jsonplaceholder.typicode.com
```

Description:
JSONPlaceholder is a fake REST API used for testing and learning purposes. It simulates resources such as users, posts, comments, albums, and todos.

Endpoints Tested

```
/users
/posts
/comments
/albums
/todos
```

Testing was performed using **GET requests only**, ensuring ethical and read-only interaction.

---

3. Tools Used

Tool	Purpose
Postman	API request testing

Tool	Purpose
Browser DevTools	Inspect headers and responses
Documentation review	Understanding API structure

---

4. Methodology

The following steps were used to perform the security analysis:

1. Review API documentation
2. Identify available endpoints
3. Send requests using Postman
4. Inspect request headers and responses
5. Check authentication requirements
6. Analyze possible security risks
7. Classify risk severity
8. Suggest remediation steps

---

5. Identified Security Risks

Risk 1: Open API Endpoints

Endpoint Example

```
GET /users
GET /posts
```

Observation

The API allows access to data without requiring authentication tokens such as:

- API key
- OAuth token
- JWT

Anyone can retrieve user and post information.

Risk Severity: Medium

Business Impact

If this were a production API, attackers could easily collect sensitive user data or scrape the platform's data without authorization.

Remediation

Implement authentication mechanisms such as:

- API keys
- OAuth 2.0
- JWT tokens

Access to user data should require authentication.

Risk 2: Excessive Data Exposure

Endpoint

```
GET /users
```

Observation

The API response exposes multiple user attributes such as:

- Name
- Email
- Phone
- Address
- Company details

Example response snippet:

```
{
 "name": "Leanne Graham",
 "email": "leanne@example.com",
 "phone": "1-770-736-8031",
 "address": {...}
}
```

Risk Severity: Medium

Business Impact

Exposing unnecessary data increases the risk of:

- privacy breaches
- social engineering attacks
- data scraping

Remediation

Return only required fields.
Use **data filtering or response minimization**.

Example:

```
GET /users?fields=name,email
```

Risk 3: Missing Rate Limiting

Observation

The API does not show evidence of request throttling.

Attackers could repeatedly send requests to endpoints.

Risk Severity: Medium

Business Impact

Without rate limiting, attackers could:

- overload the API
- cause service disruption
- perform large-scale scraping

Remediation

Implement rate limiting such as:

```
100 requests per minute per user
```

Also use:

- API gateway throttling
- request quotas

Risk 4: Lack of Input Validation

Example Test

```
GET /posts?id=abc
```

Observation

The API accepts unusual input without strong validation.

Risk Severity: Low

Business Impact

Poor input validation could lead to:

- injection attacks
- unexpected server behavior
- application errors

Remediation

Implement strict validation:

- numeric IDs
- proper query parameter validation
- server-side filtering

---

6. Risk Summary Table

Risk	Severity	Impact
Open endpoints	Medium	Unauthorized data access
Excessive data exposure	Medium	Privacy risk
Missing rate limiting	Medium	API abuse
Weak input validation	Low	Potential injection

Screenshots

In **Postman**, run:

```
https://jsonplaceholder.typicode.com/users
```

---

7. Conclusion

This API Security Risk Analysis demonstrates common issues that could appear in real-world APIs. While the tested API is designed for learning, similar patterns in production environments could lead to serious security vulnerabilities.

Organizations should implement strong authentication, rate limiting, proper input validation, and minimal data exposure to secure their APIs.